# Experiment No. 5

# Feature Extraction and Image Analysis using SIFT and HOG

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
# 1. Load Image
image_path = "image.jpg.png"
image = cv2.imread(image_path)
if image is None:
    print("Image not found! Please check the image path.")
else:
    print("Image loaded successfully!")
    # 2. Convert BGR to RGB
    rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    # Convert image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Display original and grayscale image
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(rgb_image)
    plt.title("Original Image")
    plt.axis("off")
    plt.subplot(1, 2, 2)
    plt.imshow(gray, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")
    plt.show()
    # 3. SIFT Feature Extraction
    sift = cv2.SIFT_create()
    # Detect keypoints and compute descriptors
    keypoints, descriptors = sift.detectAndCompute(gray, None)
    print("\nSIFT Feature Extraction")
    print("-----------------------")
    print("Number of Keypoints:", len(keypoints))
    if descriptors is not None:
        print("Descriptor Shape:", descriptors.shape)
    # Draw SIFT keypoints
    sift_image = cv2.drawKeypoints(
        rgb_image,
        keypoints,
        None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )
    # Display SIFT keypoints
    plt.figure(figsize=(8, 6))
    plt.imshow(sift_image)
    plt.title("SIFT Keypoints")
    plt.axis("off")
    plt.show()
```

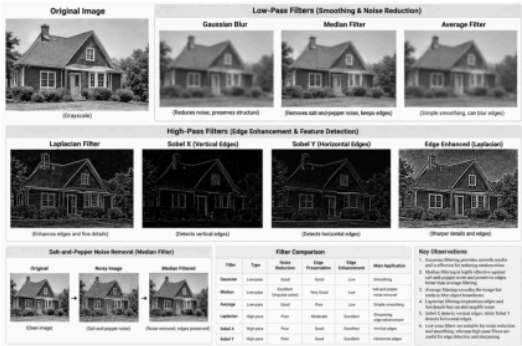
```
    # 4. HOG Feature Extraction
    hog_features, hog_image = hog(
        gray,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        block_norm="L2-Hys",
        visualize=True
    )
    print("\nHOG Feature Extraction")
    print("----------------------")
    print("Number of HOG Features:", len(hog_features))
    print("HOG Feature Shape:", hog_features.shape)
    # Improve HOG visualization
    hog_image_rescaled = exposure.rescale_intensity(
        hog_image,
        in_range=(0, 10)
    )
    # Display HOG image
    plt.figure(figsize=(8, 6))
    plt.imshow(hog_image_rescaled, cmap="gray")
    plt.title("HOG Descriptor Visualization")
    plt.axis("off")
    plt.show()
    # 5. Compare SIFT and HOG
    print("\nComparison of SIFT and HOG")
    print("--------------------------")
    print("SIFT Keypoints:", len(keypoints))
    print("SIFT Descriptor Dimension:",
          descriptors.shape if descriptors is not None else "None")
    print("HOG Feature Dimension:", len(hog_features))
    print("\nSIFT:")
    print("- Detects distinctive keypoints")
    print("- Robust to scale and rotation changes")
    print("- Useful for image matching and object recognition")
    print("\nHOG:")
    print("- Represents local gradient orientations")
    print("- Captures shape and edge information")
    print("- Useful for object detection and recognition")
```

```
Image loaded successfully!
```

Original Image



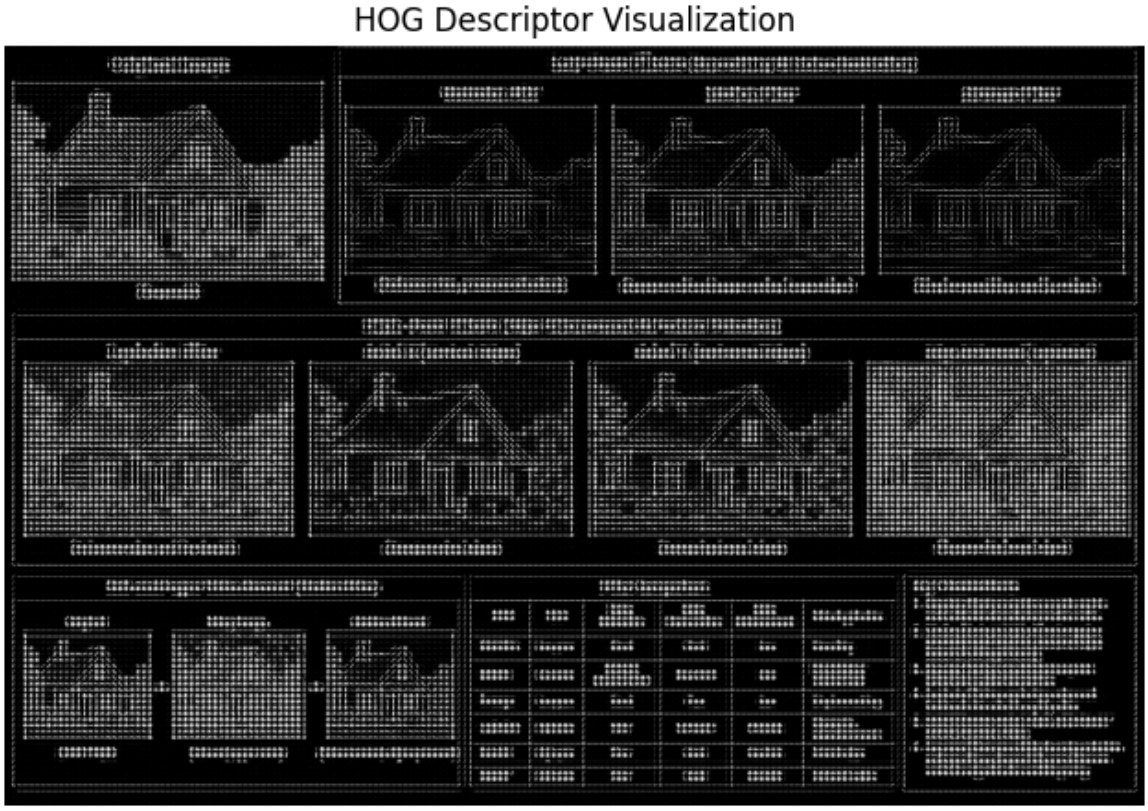

Grayscale Image




```
SIFT Feature Extraction
-----------------------
Number of Keypoints: 9437
Descriptor Shape: (9437, 128)
```




```
HOG Feature Extraction
----------------------
Number of HOG Features: 873252
HOG Feature Shape: (873252,)
```



```
Comparison of SIFT and HOG
--------------------------
SIFT Keypoints: 9437
SIFT Descriptor Dimension: (9437, 128)
HOG Feature Dimension: 873252

SIFT:
- Detects distinctive keypoints
- Robust to scale and rotation changes
- Useful for image matching and object recognition

HOG:
- Represents local gradient orientations
- Captures shape and edge information
- Useful for object detection and recognition
```

# QUESTIONS

**1. What is feature extraction, and why is it important in computer vision?**

Feature extraction is the process of identifying important information from an image.

It is important because:

- Reduces image data
- Identifies important patterns
- Helps image matching
- Improves object recognition and classification

Examples: **SIFT, HOG, SURF, ORB**

**---**

**2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).**

SIFT detects and describes important image features.

Main steps:

1. **Scale-Space Construction** – Creates images at different scales.
2. **Keypoint Detection** – Detects keypoints using DoG.
3. **Keypoint Localization** – Removes unstable keypoints.
4. **Orientation Assignment** – Assigns orientation to keypoints.
5. **Feature Descriptor** – Creates a 128-dimensional descriptor.
6. **Feature Matching** – Compares descriptors between images.

**---**

**3. What are keypoints and feature descriptors in image analysis?**

**Keypoints** are distinctive locations in an image, such as corners, edges, and blobs.

**Feature descriptors** are numerical representations of the regions around keypoints.

| Keypoints | Feature Descriptors |
|---|---|
| Identify important locations. | Describe local regions. |
| Represent image positions. | Represent image information numerically. |
| Used to locate features. | Used for matching and recognition. |

---

**4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.**

HOG represents an image using the distribution of **edge directions and gradients**.

Main steps:

1. Calculate gradients.
2. Find magnitude and orientation.
3. Divide image into cells.
4. Create orientation histograms.
5. Normalize blocks.
6. Create the final feature vector.

HOG is mainly useful for **shape, edge, and object detection**, especially human detection.

---

**5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.**

| Feature | SIFT | HOG |
|---|---|---|
| Main purpose | Local features | Shape and edges |
| Robustness | Scale and rotation robust | More sensitive to changes |
| Feature type | Keypoint-based | Gradient-based |
| Scale invariant | Yes | No |
| Complexity | Relatively high | Generally lower |
| Applications | Image matching, recognition | Human/object detection |

---

**6. Why is SIFT considered invariant to scale and rotation?**

SIFT is **scale invariant** because it detects keypoints at different scales.

It is **rotation invariant** because an orientation is assigned to each keypoint.

Therefore, SIFT can match features even when the **size or orientation of an object changes**.

---

**7. Mention three real-world applications where HOG descriptors are commonly used.**

Three applications are:

1. **Human Detection** – Detects people in images and videos.
2. **Pedestrian Detection** – Used in surveillance and transportation systems.
3. **Object Detection** – Detects objects using shape and edge patterns.

---

**8. Why is feature extraction performed before image classification or object detection?**

Feature extraction converts raw image pixels into **meaningful features**.

It helps to:

- Reduce data size
- Identify important patterns
- Capture edges and shapes
- Improve classification
- Increase computational efficiency

---

**9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?**

| Handcrafted Features | Deep Learning Features |
|---|---|
| Designed manually. | Learned automatically. |
| Need less training data. | Usually need large datasets. |
| Lower computational requirements. | Higher computational requirements. |
| Easier to interpret. | More difficult to interpret. |
| Examples: SIFT, HOG. | Example: CNN features. |

**Advantages:** Simple, interpretable, and useful for specific tasks.

**Limitations:** Less effective for complex datasets and cannot automatically learn new features.

---

**10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?**

Feature extraction converts images into **useful feature representations**.

- **Image Matching:** Finds corresponding features between images.
- **Face Recognition:** Extracts facial shapes, edges, and textures.
- **Object Detection:** Identifies object shapes and structures.

Process:

```
Input Image
     ↓
Preprocessing
     ↓
Feature Extraction
     ↓
Feature Representation
     ↓
Classification / Matching / Detection
     ↓
Final Result
```